KodiKit Operations Guide

Operating a self-updating Kodi add-on repository, and provisioning a Fire TV Stick 4K from it.

This document is the operations manual for the KodiKit repository in this directory, and the provisioning procedure for the Fire TV Stick it targets.

Book One covers the repository: what each file does, how the build pipeline works, how add-ons reach a device, and how updates keep landing without anyone touching the box.

Book Two provisions a Fire TV Stick 4K (2018, model E9L29Y) from that repository, with the numbers that hardware actually needs.

The design decision behind both: this is a **repository, not a build**. A build cannot update itself and loses your settings every time you reinstall it. A repository is what Kodi natively polls, and the one-click convenience of a build is delivered by an add-on inside it — the Toolbox. Book Two, Appendix B makes the full argument, and covers making a snapshot anyway if your situation calls for it.

> **Installation order is part of the procedure, not an appendix.** Book One, Part 10 is the deployment sequence; Book Two, Part 4 applies it to the stick. Order is the difference between a working box and one whose subtitle service never becomes the default.

Contents

BOOK ONE

Running the repository

This book documents the repository in this directory: what each file does, how the build pipeline works, and how to run it. It assumes the repo exists — it is an operations manual, not a from-scratch tutorial.

The design decision behind it: **this is a repository, not a build.** A build is a snapshot of somebody's `userdata` and `addons` folders. It cannot update itself, reinstalling it wipes your configuration, and it breaks on every Kodi release. A repository is the mechanism Kodi natively polls, so every add-on inside it updates individually, forever. The one-click convenience people actually want from a build is provided by an add-on *inside* the repository — the Toolbox — which installs the whole curated set in one pass. Book Two covers using it.

Part 0 — How a Kodi repository works

Four moving parts.

Piece	What it is	Where
Add-on zip	An add-on folder, zipped, named `id-version.zip`	`docs/zips/<id>/`
Index	`addons.xml` — every add-on's `addon.xml` concatenated	`docs/addons.xml`
Checksum	MD5 of the index	`docs/addons.xml.md5`
Repository add-on	A tiny add-on holding the URLs to the three above	`docs/repository.kodikit.zip`

The update loop:

1. Kodi periodically downloads `addons.xml.md5`. It is 32 bytes, so this is cheap.
2. If that hash differs from last time, it downloads the full `addons.xml`.
3. It compares each `version=` against what is installed.
4. Anything newer is downloaded from `docs/zips/` and installed.

So automatic updates reduce to one rule: **when an add-on changes, its version must increase and the index must be regenerated.** `scripts/build_repo.py` does both, and CI runs it daily.

> **The most common failure in any Kodi repo.** Files change but `version=` does not. Kodi concludes it is already current and never downloads the change. Nothing in the logs looks wrong. The builder here avoids this for mirrored add-ons by reading the version from upstream's own `addon.xml`, but for local add-ons in `src/` you must bump it yourself.

Part 1 — Prerequisites

Requirement	Notes
Python 3.8+	Runs the builder. No third-party packages except Pillow for artwork.
Git + a GitHub account	The repo must be public — Kodi has no credential handling worth using.
Network access	The builder calls the GitHub API and `mirrors.kodi.tv`.
Kodi 21 (Omega) or 22 (Piers)	What the dependency checker validates against.

Part 2 — Layout

```
kodi-repo/
■■■ addons.json            THE CURATED LIST - what to mirror and what to install
■■■ repo.config.json       Repo identity: name, ids, GitHub target, hosting
■■■ scripts/
■   ■■■ build_repo.py      Sync mirrors, package, build repo addon, write index
■   ■■■ check_deps.py      Verify every <import> resolves. Hard-fails CI.
■   ■■■ make_art.py        Regenerate icon.png / fanart.png (needs Pillow)
■■■ src/
■   ■■■ script.kodikit.toolbox/   The one-click installer + maintenance addon
■■■ assets/                Committed icon and fanart
■■■ docs/                  PUBLISHED TREE - the only part Kodi ever reads
■   ■■■ addons.xml         GENERATED index
■   ■■■ addons.xml.md5     GENERATED checksum; the update trigger
■   ■■■ index.html         GENERATED landing page
■   ■■■ repository.kodikit.zip     Stable-URL copy of the repo addon
■   ■■■ zips/<id>/<id>-<version>.zip (+ .md5)
■■■ guides/                This documentation
```

What you edit versus what is generated

Confusing these is the classic mistake. Everything under `docs/` is regenerated on every build; hand-edits are lost, and editing `addons.xml` directly breaks the checksum and invalidates the whole repository.

Path	Who writes it	Edit by hand?
`addons.json`, `repo.config.json`	You	**Yes — this is the control surface**
`src/**`	You	Yes
`scripts/**`	You	Rarely
`docs/**`	`build_repo.py`	**Never**

`docs/` is committed even though it is generated, because that is what GitHub serves to Kodi. That is the whole delivery mechanism.

Why `docs/`

The published tree is called `docs/` because GitHub Pages can serve a `/docs` folder directly. It works identically over raw URLs without Pages enabled, which is the current default — see Part 7.

Part 3 — Configuration

Everything about the repo's identity lives in `repo.config.json`:

Field	Current value	What it controls
`repo_id`	`repository.kodikit`	Add-on ID of the repository itself. Globally unique, permanent.
`repo_name`	`KodiKit`	Display name in Kodi's UI. Cosmetic.
`repo_version`	`1.0.0`	Bump when the repo add-on's own URLs or metadata change.
`provider`	`StrainBrainOfficial`	Shown as the author. Cosmetic.
`github_user` / `github_repo`	`StrainBrainOfficial` / `kodi-repo`	**Baked into every URL.** Must match where you actually push.
`branch`	`main`	Branch the URLs point at.
`hosting`	`raw`	`raw` or `pages` — see Part 7.
`keep_versions`	`2`	Old zips retained per add-on.
`toolbox_id`	`script.kodikit.toolbox`	Which local add-on the Toolbox is.

Renaming the repo: edit `repo_id`, `repo_name`, `github_user`, `github_repo`, then rebuild. To rename the Toolbox too, rename `src/script.kodikit.toolbox/`, update the `id` in its `addon.xml`, and update `toolbox_id`.

> `github_user` and `github_repo` are concatenated into the URLs inside the repository add-on, which then ships to every device. Getting them wrong yields a repository that installs cleanly and then finds nothing — with no useful error. Push to exactly the path configured here.

Part 4 — What is in the curated set

The set is deliberately tools and utilities only. No streaming or scraper add-ons: those are what get repositories taken down, and they break every few weeks. This one is built to still work in a year untouched.

Add-ons reach a device by one of three routes, and the distinction matters:

1. Mirrored here (3). Packaged into this repo because Kodi's official repo does not carry them. Re-synced from upstream daily.

ID	What it does
`service.subtitles.a4ksubtitles`	Multi-source subtitle search — OpenSubtitles, Addic7ed, Podnapisi, SubDL, SubSource
`script.black.bars.never`	Detects and removes black bars during playback, **including hardcoded ones**
`script.service.janitor`	Deletes watched video files on a schedule to reclaim disk

2. Installed from Kodi's official repo (17). The Toolbox installs these *by ID*, so they stay on the official update channel — which is where you want them. Includes `inputstream.adaptive`, `inputstream.ffmpegdirect`, `script.trakt`, `service.upnext`, `script.kodi.loguploader`, `plugin.video.youtube`, and optional entries like `skin.copacetic`, `script.plexmod` and `pvr.iptvsimple`.

3. From a vendor's own repo (2). Listed in the README, not installed automatically, because they resolve dependencies the official repo cannot: TheMovieDb Helper 6.x and Jellyfin for Kodi.

> **Why route 2 exists.** Mirroring an add-on that Kodi's official repo already carries would fork it onto your update channel and make you responsible for tracking it. Installing by ID keeps it on the maintainer's channel. Only mirror what is genuinely unavailable.

Part 5 — Adding an add-on

This is the procedure for putting any add-on you want into the repository. Six steps, and step 2 is the one that silently breaks things if you skip it.

5.1 Decide which route it takes

Every add-on reaches a device by exactly one of three routes. Picking the wrong one either forks maintenance onto you or ships something that cannot install.

```
Is it already in Kodi's official repo?
  YES -> "official"   installed by ID; stays on the maintainer's update channel
  NO  |
      Does it install cleanly with only dependencies that exist somewhere?
        YES -> "mirror"     packaged into this repo from its GitHub source
```

```
                NO  -> "external"   documented only; the vendor's own repo resolves it
```

Answer the first question with a command rather than a guess:

```
        python3 - <<'EOF'
        import gzip, re, urllib.request
        url = "https://mirrors.kodi.tv/addons/omega/addons.xml.gz"
        req = urllib.request.Request(url, headers={"User-Agent": "kodikit"})
        raw = gzip.decompress(urllib.request.urlopen(req, timeout=60).read()).decode("utf-8",
        "replace")
        ids = set(re.findall(r'<addon\s+id="([^"]+)"', raw))
        for probe in ["script.trakt", "service.vpn.manager"]:
            print("%-34s %s" % (probe, "IN official" if probe in ids else "NOT in official"))
        EOF
```

Put your candidate IDs in the `probe` list. `IN official` means use the `official` section — mirroring it would fork an add-on onto your update channel and make you responsible for tracking it, for no benefit.

> **The `external` route is not a failure.** It exists for add-ons whose dependencies no official repo ships. TheMovieDb Helper 6.x hard-requires `script.module.pil`, a compiled Pillow build nobody publishes; mirroring it would ship an add-on that cannot install. The dependency checker in Part 6 is what tells you this, before your users find out.

5.2 Find the add-on ID

The ID is not the display name, and getting it wrong is the most common mistake. Three reliable sources:

Where	How
The source repo	Open its `addon.xml` and read the `id=` attribute of the root `<addon>` element
An installed copy	The folder name under `~/.kodi/addons/`, e.g. `script.trakt/`
Kodi itself	Add-ons → the add-on → Information

The ID in `addons.json` must match the `id=` in the add-on's own `addon.xml` exactly. For mirrored add-ons the builder uses it to *find* the add-on inside the upstream tarball, so a wrong ID means the sync reports it as missing.

5.3 The three entry shapes

`addons.json` has three sections. Add your entry to exactly one.

`mirror` — packaged into this repo from GitHub:

```
        {
          "id": "service.vpn.manager",
          "name": "VPN Manager for OpenVPN",
          "category": "privacy",
          "why": "Connects and maintains an OpenVPN tunnel from inside Kodi.",
```

```
      "github": "Zomboided/service.vpn.manager",
      "track": "release",
      "optional": true
    }
```

`official` — installed by ID from Kodi's own repo. No `github`, no `track`:

```
    {
      "id": "script.trakt",
      "name": "Trakt",
      "category": "tracking",
      "why": "Scrobbling, watched-state sync and lists across devices."
    }
```

`external` — documented, never auto-installed. Different shape: no `id`, a vendor repo `url`, and the add-ons it provides:

```
    {
      "name": "TheMovieDb Helper (6.x)",
      "why": "Hard-requires script.module.pil, which no official repo ships.",
      "url": "https://jurialmunkey.github.io/repository.jurialmunkey/",
      "addons": ["plugin.video.themoviedb.helper", "script.skinvariables"]
    }
```

Field	Applies to	Meaning
`id`	mirror, official	The add-on ID. Must match its `addon.xml`.
`name`	all	Display name in the Toolbox picker
`category`	mirror, official	Free-form grouping; sorts the picker. In use: `subtitles`, `playback`, `tracking`, `tools`, `interface`, `privacy`.
`why`	all	Prose justification. Ends up in the README — write it for your future self.
`github`	mirror	`owner/repo`
`track`	mirror	`release` (newest release, falling back to the branch head) or `branch` (always the head)
`ref`	mirror	Pin a branch, e.g. `"ref": "matrix"`
`optional`	mirror, official	`true` leaves it **unticked** in the picker. Use for anything needing credentials, a subscription, or extra hardware.

5.4 The procedure

1. **Edit `addons.json`.** Add the entry to the right section.

2. **Bump the Toolbox version** in `src/script.kodikit.toolbox/addon.xml` — patch for adding or dropping an entry, minor for a behaviour change.
3. **Build:** `python3 scripts/build_repo.py`
4. **Check dependencies:** `python3 scripts/check_deps.py`
5. **Commit and push.** CI re-runs both and commits the regenerated `docs/`.
6. **Verify** with 5.5.

> **Why step 2 is mandatory, and what happens without it.** The curated list ships *inside* the Toolbox — the builder copies `addons.json` into the add-on when it packages it. So adding an entry changes the Toolbox's contents but not its version, and Kodi keys updates off the version in `addons.xml`. The index stays byte-identical, its checksum does not change, and **no device ever fetches the new list.** The build looks perfectly successful.

>

> `build_repo.py` refuses to finish in this state: if a local add-on's zip contents change while its version does not, it names the stale add-on and exits non-zero. If you see that error, bump the version and rebuild — that is the whole fix.

Adding to `mirror` also publishes the add-on itself, so devices can install it directly. The version bump is what makes it *appear in the Toolbox picker*.

5.5 Verify it landed

```
# The add-on is in the published index
grep -o 'id="service.vpn.manager" [^>]*version="[^"]*"' docs/addons.xml

# Its zip exists and is Kodi-shaped (id/addon.xml at the root)
unzip -l docs/zips/service.vpn.manager/*.zip | head -5

# The Toolbox carries the new list
python3 -c "import zipfile,json,glob; z=sorted(glob.glob('docs/zips/script.kodikit.too
lbox/*.zip'))[-1]; \
m=json.loads(zipfile.ZipFile(z).read('script.kodikit.toolbox/resources/addons.json'));
 \
print(sorted(e['id'] for s in ('mirror','official') for e in m[s]))"
```

On a device: Toolbox → Install curated add-ons. The new entry appears in the picker, unticked if you marked it `optional`.

5.6 When a mirror sync fails

The builder keeps the previous zip and reports the failure rather than publishing something broken.

Symptom	Cause	Fix
`no published release, falling back to branch head`	Upstream publishes no releases	Normal. Set `"track": "branch"` to make it explicit.
Add-on not found in the tarball	`id` does not match the upstream `addon.xml`	Correct the `id`
Sync fails after an upstream rename	Default branch changed	Set `"ref"` explicitly
`check_deps.py` reports an unresolvable import	A dependency nobody publishes	Move it to `external` and document the vendor repo
Upstream is a monorepo	—	Not a problem; the builder scans for the matching `addon.xml` at any depth

5.7 Removing an add-on

Delete its entry, bump the Toolbox version, rebuild, push. Its zips remain in `docs/zips/` until pruned by `keep_versions`, and copies already installed on devices keep working but stop receiving updates. To force one off a device, uninstall it there — a repository cannot retract an add-on.

Part 6 — The build pipeline

```
python3 scripts/build_repo.py && python3 scripts/check_deps.py
```

`build_repo.py` runs four stages:

1. Sync mirrored add-ons. For each `github` entry, ask the GitHub API for the newest release (or branch head), download the tarball, find the add-on folder, and repackage it as a Kodi-shaped zip with `<id>/addon.xml` at the root. Versions already present are skipped, so a quiet day produces no output and no commit.

2. Package local add-ons. Everything in `src/` is zipped at the version in its `addon.xml`. **This is where you must have bumped the version yourself** — the builder packages what it is told.

3. Build the repository add-on. Generates `addon.xml` from `repo.config.json`, zips it into `docs/zips/`, and writes a stable copy at `docs/repository.kodikit.zip` so the install URL never changes with version.

4. Regenerate the index. Collects the newest zip per add-on, concatenates their `addon.xml` bodies into `docs/addons.xml`, writes the MD5, prunes to `keep_versions`, and regenerates `docs/index.html`.

Zips are written with **fixed timestamps**, so identical content produces byte-identical output. Without this, every run would produce a different zip and CI would commit noise daily.

The dependency checker

`check_deps.py` fetches Kodi's official `addons.xml.gz` for **Omega and Piers**, unions it with this repo's own IDs, and verifies every non-optional `<import>` in the index resolves against at least one. It **fails the build** if not.

```
checked 5 add-ons against: omega, piers
all dependencies resolve
```

This is not ceremony. An unresolvable dependency makes Kodi refuse to install the add-on, and the failure appears on the user's device, not in your build. It has already earned its place: TheMovieDb Helper 6.x hard-requires `script.module.pil`, a compiled Pillow module **no official Kodi repo ships**. Mirroring it would have shipped an add-on that cannot install. The checker caught it, so 6.x is documented under the vendor-repo route and the dependency-clean 5.x is in the one-click list instead.

If the official index is unreachable the check warns and passes rather than blocking on a network failure.

Part 7 — Publishing

Two hosting modes, set by `hosting` in `repo.config.json`.

Mode	URL form	Notes
`raw` (current)	`raw.githubusercontent.com/USER/REPO/main/docs`	Works immediately after push, no setup. ~5 min cache.
`pages`	`USER.github.io/REPO`	Better caching. Requires enabling Pages for `/docs`.

Switching modes changes the URLs baked into the repository add-on, so **existing users must reinstall the repo zip once.** Choose before distributing.

Verify after pushing:

```
curl -sI https://raw.githubusercontent.com/USER/REPO/main/docs/addons.xml | head -1
curl -s  https://raw.githubusercontent.com/USER/REPO/main/docs/addons.xml.md5
```

The first must return `200`; the second a bare 32-character hex string.

Part 8 — Automation

`.github/workflows/sync.yml` runs daily at 05:15 UTC, on manual dispatch, and on pushes that touch `addons.json`, `repo.config.json`, `src/`, `scripts/`, or the workflow itself.

It runs the builder, then the dependency checker, and commits `docs/` back to the branch only if something changed. A `concurrency` group prevents overlapping runs from fighting over the same commit.

The full unattended chain:

```
upstream publishes  ->  05:15 UTC sync job packages the new version
```

```
                              ->  check_deps.py verifies it can actually install
                              ->  docs/ committed, addons.xml.md5 changes
                              ->  Kodi notices within 24h and installs silently
```

Worst case is about 48 hours from upstream release to device.

Part 9 — Making sure updates actually land

Three things must line up.

Kodi must be set to auto-update. Settings → Add-ons → Updates → *Install updates automatically*. In `guisettings.xml` this is `general.addonupdates` = `0` (`1` notifies, `2` never). There is also a per-add-on override in Add-on info → Auto-update; if one was ever set to Never, the global setting will not rescue it.

The version must increase. Kodi only moves upward. Re-uploading a zip with the same version does nothing regardless of content, and lowering a version to force a change does nothing either. A broken release is fixed with a **higher** patch version, never a re-upload.

The checksum must change. That is what Kodi polls. The builder handles it; the only way to break it is editing `docs/addons.xml` by hand afterwards.

To force an immediate check on a device: Add-ons → My add-ons → select the repository → Check for updates. Remotely, toggling the repository off and on forces a re-scan on the next cycle:

```
curl -s -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"Addons.SetAddonEnabled",
       "params":{"addonid":"repository.kodikit","enabled":true}}' \
  http://DEVICE-IP:8080/jsonrpc
```

If an update does not land, `kodi.log` is definitive:

```
adb shell "grep -i 'repository.kodikit\|CRepository' \
  /sdcard/Android/data/org.xbmc.kodi/files/.kodi/temp/kodi.log | tail -40"
```

`CRepository::FetchIndex` failures point at URLs or checksums; "no update available" against a bumped version points at a stale index.

Part 10 — Installing on a device, in order

Order is part of the procedure, not a detail. Four rules drive it:

Rule	Consequence
Config files are read at startup	`advancedsettings.xml` must exist before Kodi is used
InputStream add-ons underpin playback	They come before anything that plays video
Declared dependencies resolve automatically	Never hand-install `script.module.*`
Services claim defaults on first run	Subtitles must be in place before you set the default

Phase A — Foundation

#	Step
1	Install Kodi, launch once, quit
2	Settings → System → Add-ons → **Unknown sources** → **On**
3	Settings → Add-ons → Updates → **Install updates automatically**
4	Settings → File manager → Add source → the URL from Part 7, named `kodikit`
5	Add-ons → Install from zip file → `kodikit` → `repository.kodikit.zip`
6	Add-ons → Install from repository → KodiKit → install **KodiKit Toolbox**

Step 3 goes before any add-on exists so every one inherits it. Step 5 matters more than it looks: an add-on sideloaded from a loose zip belongs to no repository, so Kodi has nowhere to check for updates and it stays at its installed version forever.

Phase B — The Toolbox does the rest

Open the Toolbox → **Install curated add-ons**. It multi-selects the whole set, filters out anything already installed, and leaves optional entries unticked. Kodi resolves each add-on's declared dependencies automatically and in the right order, which is why there is no manual InputStream step here — installing the parents pulls them.

Phase C — Tune and finish

#	Step
7	**Decide on the VPN.** VPN Manager ships in the repository and appears in the picker unticked. Either tick it and configure it now, or consciously decide against it. Do not leave it undecided — see below.
8	Toolbox → **Apply streaming cache tuning** → pick your device profile → restart
9	Set the subtitle default: Settings → Player → Language → a4kSubtitles. **An unset default is why "subtitles don't work".**
10	Toolbox → **Clean up caches** to clear install debris
11	Optional: enable Settings → Services → Control → Allow remote control via HTTP, for remote diagnosis

On the VPN step. It is unticked in the installer because it needs your provider's credentials and an OpenVPN binary on the platform — it cannot be a silent default. It is a mandatory *decision* because retrofitting it later means redoing any account authorization you did on the wrong address: device-code and OAuth flows bind a session to the address that completed them, so connect the VPN **before** signing in to anything.

If you tick it: Toolbox picker → VPN Manager → then VPN Manager's own settings → provider → credentials → connect, and confirm its external-IP readout changes.

> **Check the platform and the provider before you rely on it.** VPN Manager raises an OpenVPN tunnel from inside Kodi, which needs an OpenVPN binary it can execute — fine on LibreELEC, Linux and Windows, but on Android and Fire OS an add-on generally cannot control the system VPN. It also bundles profiles for a fixed set of providers (NordVPN, ExpressVPN, PIA in 7.0.3); anything else, **Surfshark included**, uses the *User Defined* route with hand-imported `.ovpn` files and the provider's separate manual-setup credentials. On a Fire TV Stick the reliable route is the provider's own Android app, which tunnels the whole device rather than Kodi alone. Book Two, Part 4 works this through for Surfshark.

That is the whole deployment. Book Two applies it to a Fire TV Stick with device-specific numbers.

Part 11 — Maintenance

Task	When	How
Add or drop a curated add-on	Any time	Edit `addons.json`, push. CI rebuilds.
Change the Toolbox	Any time	Edit `src/`, **bump its** `addon.xml` **version**, push
Change repo identity or URLs	Rarely	Edit `repo.config.json`, bump `repo_version`, and have users reinstall the repo zip
Regenerate artwork	Rarely	`python3 scripts/make_art.py` (needs Pillow)
Check upstream health	Monthly	Read the sync job's log for skipped or failed mirrors

Part 12 — Troubleshooting

Symptom	Cause	Fix
"Could not connect to repository"	`github_user`/`github_repo` wrong, or repo private	`curl` the URL; confirm 200 and public
Repository installs, shows no add-ons	Index empty or checksum stale	Re-run the builder; never hand-edit `docs/`
"Failed to install add-on"	Zip name does not match `id-version.zip`	Re-run the builder; it names them
"Dependency not met"	Unresolvable `<import>`	Run `check_deps.py`; mirror the missing module
Add-on installs, never updates	Version not bumped	Bump it in `addon.xml` and rebuild
One add-on never updates	Per-add-on auto-update set to Never	Add-on info → Auto-update → On
CI commits every day with no real change	Zip timestamps not fixed	Already handled here; do not remove the fixed-timestamp logic
Sync job fails on one add-on	Upstream renamed a release or branch	Set `ref` explicitly in `addons.json`
Updates only appear after a restart	Normal 24h poll cycle	Force a check, or accept the cadence

BOOK TWO

Provisioning a Fire TV Stick 4K

This book provisions a Fire TV Stick using the repository from Book One. The target is the **Fire TV Stick 4K (2018, model E9L29Y)** — 1.7 GHz quad-core, 1.5 GB RAM, 8 GB storage, Android 7.1+, running Kodi 21.1. Guidance for the 1 GB gen 1/2 sticks is noted where it differs.

There is no snapshot build here, deliberately. Appendix B explains why, and covers making one anyway if your situation genuinely calls for it.

Part 1 — The hardware budget

	Stick gen 1 (2014)	Stick gen 2 (2016)	Stick 4K (2018)	Stick Lite / 3rd gen
Model	W87CUN	LY73PR	**E9L29Y**	—
RAM	1 GB	1 GB	**1.5 GB**	1 GB
Storage	8 GB (~5 usable)	8 GB (~5 usable)	**8 GB (~5 usable)**	8 GB
CPU	Dual-core 1.0 GHz	Quad-core 1.3 GHz	**Quad-core 1.7 GHz**	Quad-core 1.7 GHz
Android base	5.1	5.1	**7.1+**	9
Realistic Kodi ceiling	19–20	20	**21+**	21+

Fire OS reserves 350–450 MB for itself. Kodi with Estuary is roughly 250–300 MB. That leaves about **250 MB of headroom** on a 1 GB stick and closer to **700 MB** on the 4K. Every add-on running a background service eats into it.

Metric	1 GB sticks	Stick 4K	Unacceptable
`.kodi/` on disk	under 350 MB	under 500 MB	over 900 MB
Cold start	under 25 s	under 18 s	over 45 s
Idle RAM (PSS)	under 420 MB	under 600 MB	thrashing / restarts
Background services	4 or fewer	6 or fewer	10 or more

Storage, not RAM, is the binding constraint on the 4K. The extra 512 MB buys real room for services, but the flash is the same 8 GB, and the texture cache grows without limit until something caps it. That is what the artwork resolution settings in Appendix A are for.

> **Version matching matters more than version chasing.** Kodi 21.1 on the stick and Kodi 21.1 as the reference is the easy case. On gen 1/2 sticks (Android 5.1) the ceiling is Kodi 20 — check kodi.tv/download for the current `armeabi-v7a` minimum before committing, since it rises with each release.

Part 2 — What goes on the box

The curated set from Book One, Part 4, minus what this hardware cannot afford.

Install: the repository, the Toolbox, `inputstream.adaptive`, `inputstream.ffmpegdirect`, `service.subtitles.a4ksubtitles`, `script.black.bars.never`, `service.upnext`, `script.kodi.loguploader`, and `plugin.video.youtube`.

Consider: `script.trakt` if you want watched-state sync — it is a background service, but a light one. `script.service.janitor` only if the stick stores local recordings, which it usually does not. `service.vpn.manager` is in the repository and unticked by default — on this hardware, and with Surfshark specifically, it is the wrong route; see the VPN decision in Part 4.

Leave out on a stick:

Component	Cost
`skin.copacetic` and any third-party skin	40–150 MB plus slower menus. Estuary is the lightest maintained skin.
`script.plexmod`	Only with a Plex server; library sync is the heaviest recurring cost in the set
`pvr.iptvsimple`	Only with an M3U subscription; costs startup work otherwise
`script.extendedinfo`, `script.embuary.*`	Skin and metadata helpers that scrape in the background
Multiple subtitle services	Each is a service; one is enough

The Toolbox leaves optional entries unticked by default, so the shortest path is: accept the defaults, then untick the skin.

Part 3 — Install Kodi on the stick

1. Enable sideloading. Settings → My Fire TV → Developer Options → *Apps from Unknown Sources* → On. On newer Fire OS you may need to click the Fire TV Stick build number seven times first to reveal Developer Options.

2. Install Kodi. Either the Downloader app, or ADB from your machine:

```
adb connect FIRESTICK-IP:5555
adb install -r kodi-21.1-Omega-armeabi-v7a.apk
```

The stick is 32-bit ARM: use the `armeabi-v7a` APK, not `arm64-v8a`.

3. Launch Kodi once and quit. This creates the profile directory.

Part 4 — The provisioning run

Follow Book One, Part 10 exactly. Condensed, with the choices for this device:

#	Step	This device
1	Unknown sources → On	
2	Updates → Install updates automatically	**Do not skip.** This is what makes the stick maintenance-free.
3	Add file source	The URL from Book One, Part 7, named `kodikit`
4	Install `repository.kodikit.zip`	From the source you just added
5	Install **KodiKit Toolbox** from the repository	
6	Toolbox → **Install curated add-ons**	Untick the skin and any optional entry from Part 2
7	**Set up the VPN**	Surfshark's Fire TV app, not the Kodi add-on — see the section below. Mandatory decision; do it before step 9.
8	Toolbox → **Apply streaming cache tuning**	Choose **Fire TV Stick**. Restart when prompted.
9	Settings → Player → Language → subtitle service	Set a4kSubtitles as the default — it does nothing until you do
10	Toolbox → **Clean up caches**	Clears install debris

The tuning profile in step 7 writes a 40 MB buffer plus the artwork caps and dirty-region redraw described in Appendix A. Do not use *Balanced* here: its 64 MB buffer becomes roughly 192 MB resident, which does not fit alongside Fire OS on 1.5 GB.

The VPN decision (step 7)

Your primary VPN is **Surfshark**, and that settles how this step goes on a Fire TV Stick: use Surfshark's own Fire TV app, not the Kodi add-on.

Two independent reasons, either one sufficient:

1. **Fire OS.** VPN Manager works by executing an OpenVPN binary and raising a tunnel from inside Kodi. On Android and Fire OS an ordinary app cannot control the system VPN, so the add-on generally cannot establish one on this hardware.
2. **Provider coverage.** VPN Manager 7.0.3 bundles connection profiles for NordVPN, ExpressVPN and Private Internet Access. **Surfshark is not among them.** Using it with Surfshark means the *User Defined* route: downloading `.ovpn` config files and importing them by hand.

The route to use

#	Step
1	On the stick: Amazon appstore → install the **Surfshark** app
2	Sign in and connect. Enable **auto-connect on startup** so a reboot does not silently drop protection.
3	Optionally set **Bypasser** (Surfshark's split tunnelling) if you want specific apps outside the tunnel
4	Verify the tunnel is actually up — see below
5	Only then do any account sign-ins inside Kodi

Step 5 is the ordering that matters. Device-code and OAuth flows bind a session to the address that completed them, so signing in before the tunnel is up means redoing it later.

This route is strictly better than the add-on on this device: it covers **everything on the stick**, not just Kodi.

Verifying the tunnel

The Surfshark app reports connected, but confirm it at the OS level:

```
# A VPN tunnel interface should exist while connected
adb shell ip addr show tun0

# And Android should report an active VPN transport
adb shell dumpsys connectivity | grep -i -m5 vpn
```

If `tun0` does not exist, the app is not actually tunnelling regardless of what its UI says.

When VPN Manager is still worth ticking

Tick it in the Toolbox picker only if you also run Kodi on **Linux, LibreELEC or Windows**, where it can execute OpenVPN properly, and you want the same curated set everywhere. For Surfshark there:

- In VPN Manager settings choose **User Defined** as the provider.
- Get the `.ovpn` files from Surfshark's manual-setup page and import them via VPN Manager's *import wizard*.
- Authenticate with Surfshark's **manual-setup service credentials** from your account dashboard — these are a separate username and password, **not** your Surfshark login. Using the account email/password here is the usual reason a manual OpenVPN connection fails.

If you decide against a VPN entirely, that is a legitimate answer — just make it deliberately, and before the sign-ins.

Playback settings

The Toolbox does not touch these; set them once by hand.

Setting	Value	Why
Player → Videos → Allow hardware acceleration – **MediaCodec (Surface)**	On	Hardware decode. Without it 1080p stutters and 4K is hopeless.
Allow hardware acceleration – MediaCodec	Off	Redundant with Surface, costs memory
Adjust display refresh rate	On start/stop	Prevents judder
Sync playback to display	Off	Expensive on a weak CPU
Enable HQ scalers	0%	Pure CPU cost, no visible benefit here

Interface settings

Setting	Value
Interface → Skin	Estuary
Show RSS news feeds	Off
Startup → Perform on startup	Home screen
Media → General → Show media flags	Off
Services → Control → Allow remote control via HTTP	On, port 8080

Leaving the web server on is what makes everything in Part 5 possible without touching the device.

Part 5 — Verifying

Run these after a reboot, idle at the home screen.

```
# Memory
adb shell dumpsys meminfo org.xbmc.kodi | grep -E 'TOTAL PSS|TOTAL RSS'

# Installed size
adb shell du -sh /sdcard/Android/data/org.xbmc.kodi/files/.kodi

# Where the space actually went
adb shell "du -m /sdcard/Android/data/org.xbmc.kodi/files/.kodi/* \
  /sdcard/Android/data/org.xbmc.kodi/files/.kodi/userdata/* | sort -rn | head"

# Which services started
adb shell "grep -i 'service.*started\|Starting service' \
  /sdcard/Android/data/org.xbmc.kodi/files/.kodi/temp/kodi.log | tail -20"

# Confirm auto-update really is on
adb shell "grep addonupdates \
  /sdcard/Android/data/org.xbmc.kodi/files/.kodi/userdata/guisettings.xml"
```

Check	Pass on the Stick 4K
TOTAL PSS idle	under 600 MB
`.kodi/` size	under 500 MB
Cold start	under 18 s
Services started	6 or fewer
`general.addonupdates`	`0`
1080p / 4K playback	no dropped frames (`Ctrl+Shift+O` overlay)

A high PSS is almost always one background service. Disable them one at a time and re-measure — that is far faster than guessing.

The Toolbox's **Storage report** gives the same picture from the sofa, with no ADB, and changes nothing when you run it.

Part 6 — Keeping it small

A stick that starts at 300 MB reaches 900 MB in six months if nothing prunes it. The texture cache is nearly always the cause.

Task	Frequency	How
Storage report	Monthly	Toolbox → Storage report

Task	Frequency	How
Clean up caches	When the report looks bad	Toolbox → Clean up caches
Clean library	Occasionally	Toolbox → Clean video/music library
Re-measure idle RAM	After adding an add-on	Catch a heavy service immediately

Clearing thumbnails and the texture DB means artwork re-downloads as you browse. That is the point — but the first few minutes afterwards feel slower.

Part 7 — Troubleshooting

Symptom	Cause	Fix
Buffering on 1080p or 4K	Cache profile too large for the device	Toolbox → tuning → **Fire TV Stick**
Stutter on high-bitrate video	Hardware decode off	MediaCodec (Surface) → On
Black screen with audio	MediaCodec conflict	Disable the non-Surface MediaCodec option
Very slow menus	Third-party skin or scraping widgets	Estuary; remove widgets
Out of space within weeks	Texture cache growth	Clean up caches; confirm the artwork caps are in `advancedsettings.xml`
Subtitles never appear	Default service not set	Settings → Player → Language → a4kSubtitles
Add-ons never update	`general.addonupdates` not `0`	Set it; check the per-add-on override too
Kodi APK will not install	Wrong architecture	Use `armeabi-v7a`
Everything is slow on gen 1/2	1 GHz dual-core is the limit	Cut the add-on count; expect 1080p only. Not applicable to the 4K.

Appendix A — What the Fire TV Stick profile writes

Toolbox → Apply streaming cache tuning → Fire TV Stick produces this at `userdata/advancedsettings.xml`, backing up any existing file with a timestamp first:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<advancedsettings>
  <cache>
    <!-- 1 = buffer all internet filesystems to memory -->
    <buffermode>1</buffermode>
    <memorysize>41943040</memorysize>
    <readfactor>4</readfactor>
  </cache>

  <!-- Redraw only changed screen regions; a large win on weak GPUs. -->
  <gui>
    <algorithmdirtyregions>3</algorithmdirtyregions>
  </gui>

  <!-- Cap cached artwork. The highest-leverage setting for storage on a
       stick: the flash is 8 GB no matter how much RAM the model has. -->
  <imageres>540</imageres>
  <fanartres>720</fanartres>

  <!-- Fail fast on dead sources instead of hanging the UI. -->
  <network>
    <curlclienttimeout>20</curlclienttimeout>
    <curllowspeedtime>15</curllowspeedtime>
  </network>
</advancedsettings>
```

Why these numbers:

- `memorysize` **40 MB.** Kodi allocates roughly **three times** the configured value, so this is ~120 MB resident. The *Low memory* profile (20 MB → ~60 MB) suits 1 GB sticks; *Balanced* (64 MB → ~192 MB) is for a box with real RAM to spare.
- `imageres` **/** `fanartres`**.** Dropping fanart from 1080 to 720 more than halves texture cache growth, and on a stick feeding a TV that upscales anyway the difference is invisible. This is the single most effective setting against the 8 GB flash filling up.
- `algorithmdirtyregions` **3.** Redraw only what changed. Meaningful on this GPU.
- **Network timeouts.** Without them a dead source hangs the UI rather than failing.

Cache settings only take effect after a restart, which the Toolbox offers.

Legacy note: pre-Kodi 18 guides use `<network><cachemembuffersize>` and `<readbufferfactor>`. Those names still parse but are deprecated — the `<cache>` block above is current.

Appendix B — Why there is no snapshot build

The original goal here was a "build": a packaged `userdata` + `addons` snapshot installed by a wizard. That is the wrong shape for this, for four reasons:

1. **A build cannot update itself.** The add-ons inside it are sideloaded, belong to no repository, and therefore never receive updates. Book One's entire machinery exists to avoid exactly this.
2. **Reinstalling wipes your configuration.** Updating a build means replacing `userdata`, taking your settings with it.
3. **Builds rot on Kodi releases.** A profile built on one major version does not restore cleanly onto another; settings move and the add-on database schema changes.
4. **Builds leak credentials.** `userdata/addon_data/` holds API keys, tokens and account logins. Packaging it wholesale ships your accounts to everyone who installs it.

The Toolbox provides the part people actually want from a build — one action, whole set installed, sensible settings applied — while every add-on stays individually updatable on its own channel. A fresh stick reaches a finished state in about five minutes with Part 4, and stays current afterwards with no further attention.

If you genuinely need one

Cloning many identical devices, or a box with no reliable network, are real reasons. In that case:

```
cd ~/.kodi
zip -qr ~/mybuild-1.0.0.zip addons userdata \
  -x 'addons/packages/*' 'userdata/Database/*' 'userdata/Thumbnails/*' \
     '*/__pycache__/*' '*.pyc' '*.log' 'temp/*'
```

Exclude `Database/` and `Thumbnails/` — both rebuild themselves and are most of the size. **Audit `addon_data/` for credentials before sharing it:**

```
grep -rilE 'token|apikey|api_key|password|secret|user' ~/.kodi/userdata/addon_data/
```

Restore by extracting over `.kodi/` with Kodi **force-stopped**, not merely backgrounded — Kodi rewrites `guisettings.xml` on exit and will overwrite the restored settings otherwise:

```
adb push mybuild-1.0.0.zip /sdcard/Download/
adb shell "cd /sdcard/Android/data/org.xbmc.kodi/files/.kodi && \
  unzip -o /sdcard/Download/mybuild-1.0.0.zip"
adb shell am force-stop org.xbmc.kodi
```

Even then, install the repository afterwards so the cloned devices resume receiving updates.